



Algorithms and Data Structures

Sujeong Kim



UNIVERSITY OF
MARYLAND

Greedy Algorithms

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Optimization Problems

- An optimization problem involves making a set of choices to find the optimal solution, subject to certain constraints.
- The optimal solution is the process of finding the minimum or maximum value of an objective function.
- There may be multiple solutions that achieve the optimal value, but the optimal value itself is unique.
- Two common techniques:
 - Dynamic Programming (global)
 - Greedy Algorithms (local): choose the best option (locally optimal choices) available at each step without reconsidering previous choices.
 - Minimum spanning tree: Kruskal's algorithm, Prim's algorithm
 - Shortest paths: Dijkstra's algorithm

Intro to Greedy Algorithms

Greedy algorithms are typically used to solve optimization problems & normally consist of

- Set of *candidates*
- Set of candidates that have already been used (chosen set, partial solution)
- *Function* that checks whether a particular set of candidates *provides a solution* to the problem
- *Function* that checks if a set of candidates is *feasible*
- *Selection function* indicating at any time which is the most promising candidate not yet used
- *Objective function* giving the value of a solution; this is the function we are trying to optimize

Step by Step Approach

- Initially, the set of chosen candidates is empty
- At each step, add to this set the best remaining candidate; this is guided by selection function.
- If enlarged set is no longer feasible, then remove the candidate just added; else it stays.
- Each time the set of chosen candidates is enlarged, check whether the current set now constitutes a solution to the problem.

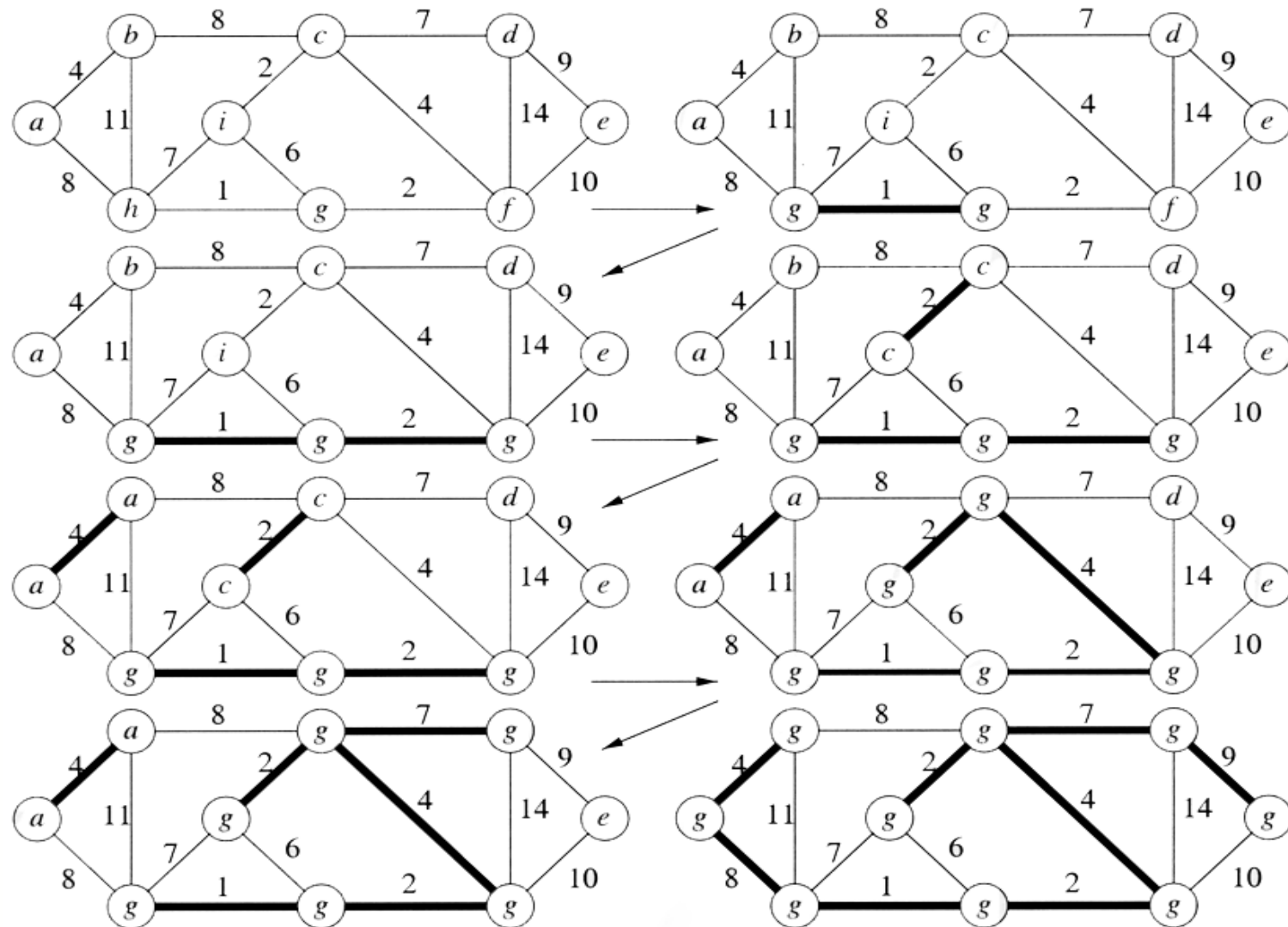
When a greedy algorithm works correctly, the first solution found in this way is always optimal.

Greedy(C)

// C is the set of all candidates

1. $S \leftarrow \emptyset$ // S is the set in which we construct solutions
2. while not *solution*(S) and $C \neq \emptyset$ do
3. $x \leftarrow$ an element of C maximizing *select*(x)
4. $C \leftarrow C \setminus \{x\}$ //remove x from the set of candidates C
5. if *feasible*($S \cup \{x\}$) then $S \leftarrow S \cup \{x\}$
6. if *solution*(S) then return S
7. else return “*there are no solutions*”

Recall: Kruskal's Algorithm



Components of Greedy Algorithms – Kruskal's MST

- Describe Kruskal's algorithm using the generic greedy-algorithm framework
 - Candidates
 - Chosen set
 - Solution test
 - Feasibility test
 - Selection rule
 - Objective

Analysis

- The selection rule is guided by the objective function. Often they are identical, but multiple plausible selection rules may exist depending on the problem
- Example: MST
 - Objective: minimize total weight
 - Selection rules:
 - Pick smallest edge (Kruskal)
 - Pick smallest edge crossing a cut (Prim)
- At every step, the procedure chooses the best morsel it can swallow, without worrying about the future. It never changes its mind: once a candidate is included in the solution, it is there for good; once a candidate is excluded, it's never considered again.

Analysis (contd.)

- Greedy algorithms do NOT always yield optimal solutions
 - Optimal for problems with the "greedy-choice property" and "optimal substructure."
 - For many complex or NP-hard problems, greedy algorithms provide good approximations but are not guaranteed to find the best solution.

Elements of Greedy Strategy

- Greedy-choice property: A global optimal solution can be arrived at by making locally optimal (greedy) choices
- Optimal substructure: an optimal solution to a problem works by including the optimal solutions to its sub-problems.
 - This principle ensures that smaller problems solved optimally lead to the best overall outcome for the main problem.
 - Be able to demonstrate that if A is an optimal solution containing s_1 , then the set $A' = A - \{s_1\}$ is an optimal solution to a smaller problem w/o s_1 . (See proof of Theorem 16.1)

Examples of Greedy Algorithms

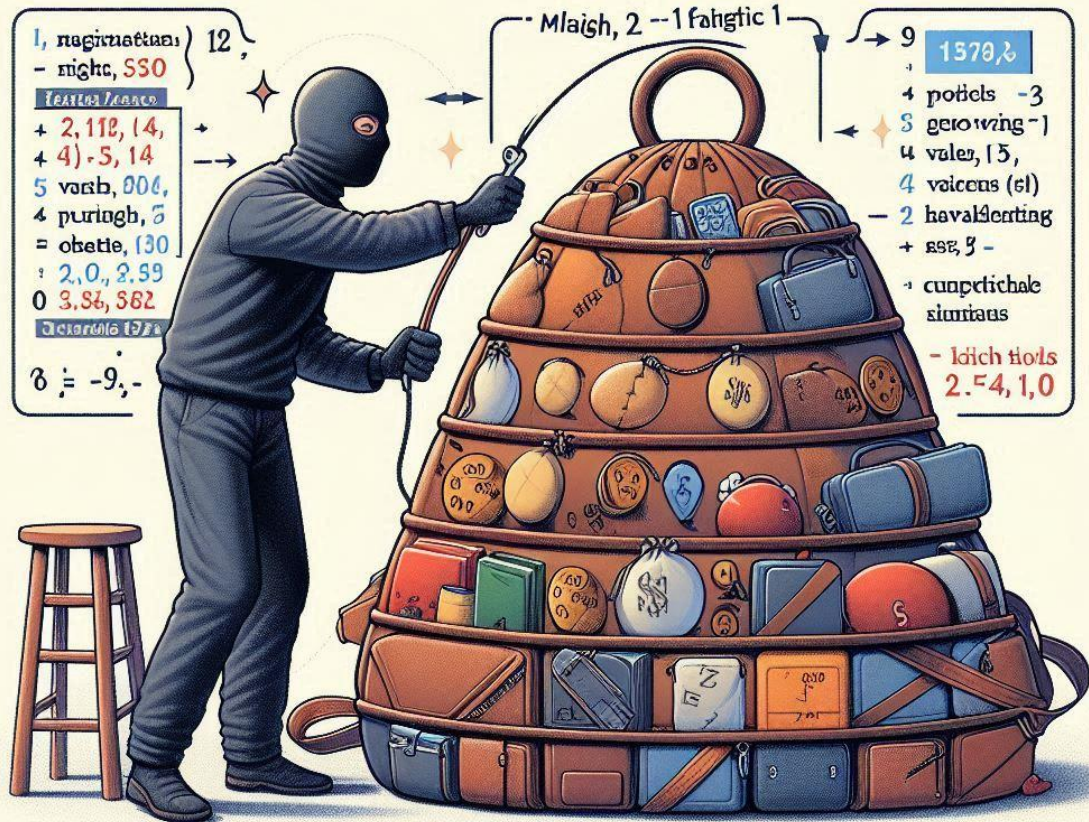
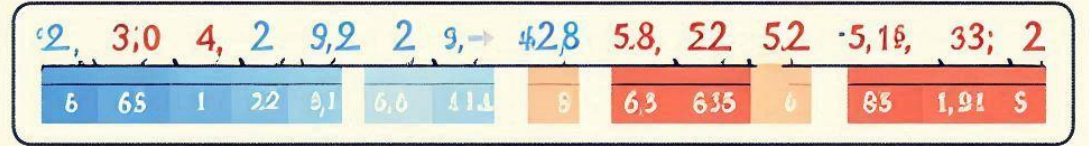
- Scheduling
 - Activity Selection (Chap 17.1)
 - Minimizing time in system
 - Deadline scheduling
- Graph Algorithms
 - Minimum Spanning Trees (Chap 24)
 - Dijkstra's (shortest path) Algorithm (Chap 25)
- Other Heuristics
 - Coloring a graph
 - Traveling Salesman (Chap 37.2)
 - Set-covering (Chap 37.3)

Optimal Greedy Algorithms

- Activity Selection:
 - Always pick the activity that finishes earliest. Guaranteed to fit the maximum number of non-overlapping activities
- Minimizing Time in System (also known as Shortest Processing Time First)
 - Perform the shortest jobs first. The average/total waiting time for all users is minimized
- Minimum Spanning Tree (Prim's/Kruskal's)
- Dijkstra's algorithm

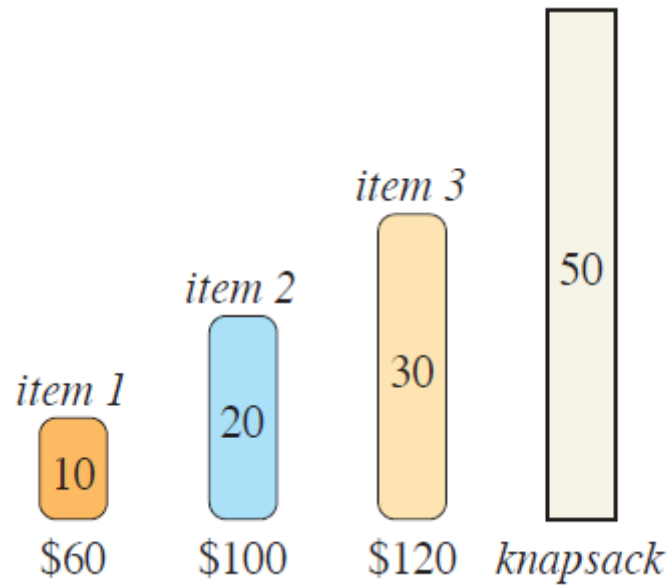
Knapsack Problem

The fractaico of thie facmerte iramucetion;



Knapsack Problem

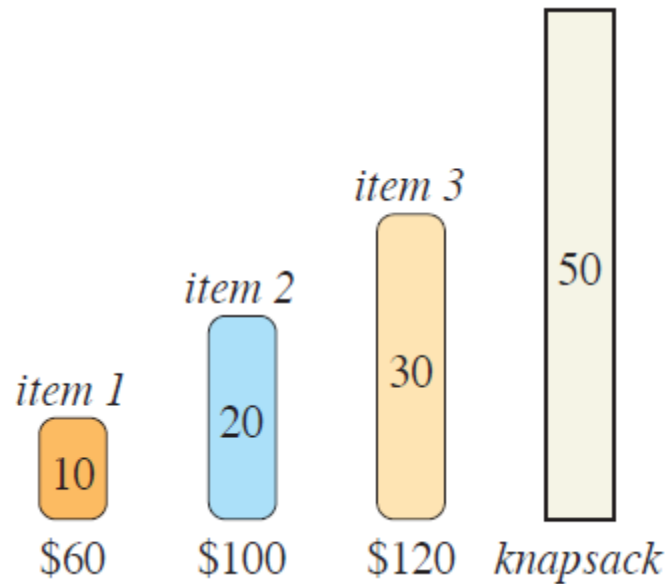
- Objective: Maximize the total value of items placed in the knapsack without exceeding the weight capacity.
- **0-1 knapsack**: Binary decision for each item (take it or leave it)



item 1	item 2	item 3	total weight	total value
0	0	0	0	0
0	0	1	30	120
0	1	0	20	100
0	1	1	50	220
1	0	0	10	60
1	0	1	40	180
1	1	0	30	160
1	1	1	60	280

Fractional Knapsack Problem

- *Fractional knapsack*: The thief can take fractions of items, instead of making a binary (0-1) choice for each item.



	Item 1	Item 2	Item 3
Weight (w_i)	10	20	30
Value (v_i)	\$60	\$100	\$120

Value per pound (v_i / w_i)	6	5	4
---------------------------------	---	---	---

What is the maximum total value and the corresponding weight?

$\frac{20}{30}$ \$80
+
20 \$100
+
10 \$60
= \$240

Knapsack Problem

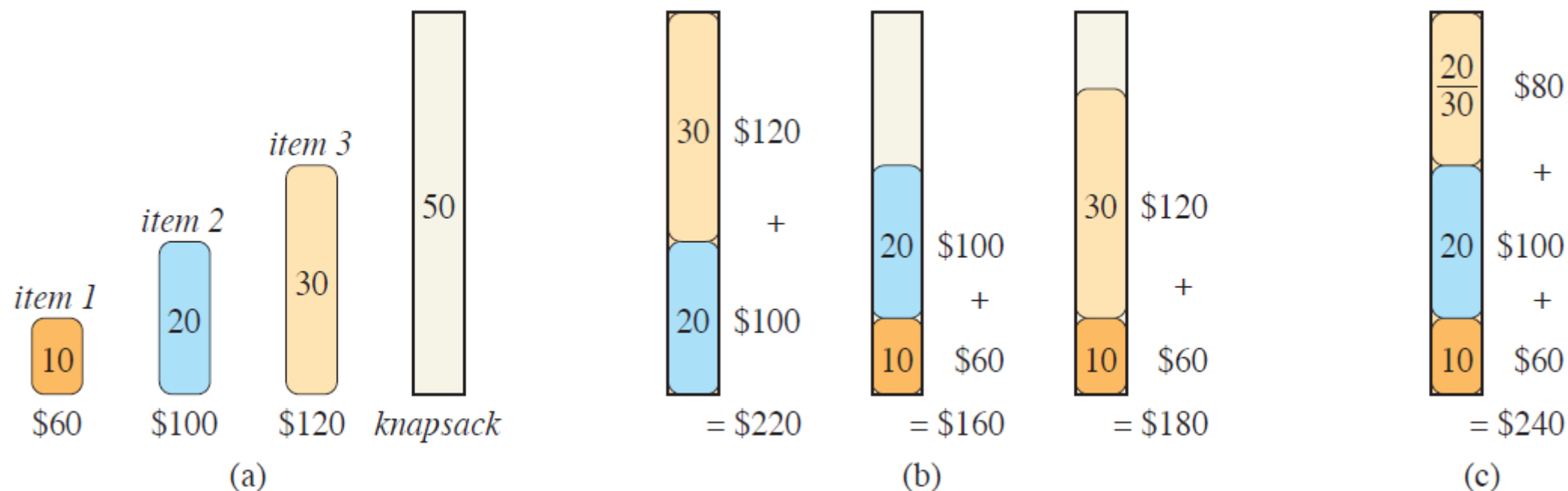


Figure 15.3 An example showing that the greedy strategy does not work for the 0-1 knapsack problem. (a) The thief must select a subset of the three items shown whose weight must not exceed 50 pounds. (b) The optimal subset includes items 2 and 3. Any solution with item 1 is suboptimal, even though item 1 has the greatest value per pound. (c) For the fractional knapsack problem, taking the items in order of greatest value per pound yields an optimal solution.

Comparisons

- Which problem exhibits greedy choice property?
- Which one exhibits optimal-substructure property?

Components of Greedy Algorithms – Fractional Knapsack

- **Candidates:** All items available for selection
- **Chosen set:** items that have been included in the knapsack
- **Solution test:** Knapsack's capacity is exactly full, or there are no candidates left
- **Feasibility test:** Checks the constraint $\sum x_i w_i \leq W$, where x_i is a fraction, w_i is the weight of the i -th item, and W is the maximum capacity of the knapsack.
- **Selection rule:** Chooses the next item (or fraction of an item) to include in the knapsack based on the highest value-to-weight ratio
- **Objective:** maximize the total value $\sum x_i v_i$

Minimizing Time in the System



Minimizing Time in the System

- A single server (a processor, a gas pump, a cashier in a bank, and so on) has n customers to serve. The service time required by each customer is known in advance: customer i will take time t_i , $1 \leq i \leq n$. We want to minimize

$$T = \sum_{i=1}^n (\text{time in system for customer } i)$$

- The “time in system” for a customer i includes the time it takes to serve all previous customers and then serve i .

Minimizing Time in the System



individual task time (minutes) for customer i	3	5	10	8	4	2
Total Time in System (minutes) for customer i	3	3+5	3+5+10	3+5+10+8	3+5+10+8+4	3+5+10+8+4+2

Minimizing Time in the System



individual task time (minutes) for customer i	t_1	t_2	t_3	t_4	t_5	t_6
Total Time in System (minutes) for customer i	t_1	$t_1 + t_2$	$t_1 + t_2 + t_3$	$t_1 + t_2 + \cdots + t_4$	$t_1 + t_2 + \cdots + t_5$	$t_1 + t_2 + \cdots + t_6$

Example

- We have 3 customers with

$$t_1 = 5, \quad t_2 = 10, \quad t_3 = 3$$

<i>Order</i>	<i>T</i>	
1 2 3:	$5 + (5+10) + (5+10+3)$	$= 38$
1 3 2:	$5 + (5+3) + (5+3+10)$	$= 31$
2 1 3:	$10 + (10+5) + (10+5+3)$	$= 43$
2 3 1:	$10 + (10+3) + (10+3+5)$	$= 41$
3 1 2:	$3 + (3+5) + (3+5+10)$	$= 29 \quad \leftarrow \text{optimal}$
3 2 1:	$3 + (3+10) + (3+10+5)$	$= 34$

Properties

- Optimal substructure
 - You have an optimal schedule for 100 customers
 - Delete the first customer
 - Is the remaining schedule the best possible way to schedule the remaining customers?
- Greedy choice
 - At any point m , the best next step is to select the customer with the minimum service time?

Designing Algorithm

- Imagine you already served a set of customers $i_1, \dots, i_m$, in that order.
- Imagine an algorithm that builds the optimal schedule step by step.
Suppose after serving customer $i_1, \dots, i_m$ we add customer j . The increase in T at this stage is

$$t_{i_1} + \dots + t_{i_m} + t_j$$

- To minimize this increase, we need only to minimize t_j . This suggests a simple greedy algorithm: at each step, add to the end of schedule the customer requiring the least service among those remaining.

Greedy(C)

// C is the set of all candidates

1. $S \leftarrow \emptyset$ // S is the set in which we construct solutions
2. while not *solution*(S) and $C \neq \emptyset$ do
3. $x \leftarrow$ an element of C maximizing *select*(x)
4. $C \leftarrow C \setminus \{x\}$
 //remove x from the set of candidates C
5. if *feasible*($S \cup \{x\}$) then $S \leftarrow S \cup \{x\}$
6. if *solution*(S) then return S
7. else return “*there are no solutions*”

// C is the set of all customers, each with a service time t_i

1. $S \leftarrow []$ // S will store the ordered schedule of customers
2. while $C \neq \emptyset$ do
3. $x \leftarrow$ an element of C minimizing t_x
4. $C \leftarrow C \setminus \{x\}$
 //remove x from the set of candidates C
5. $S \leftarrow S \cup \{x\}$
6. if S is not empty then return S
7. else return “no customers to schedule”

Optimality Proof (I)

- This greedy algorithm is always optimal.

(Proof) Let $I = (i_1, \dots, i_n)$ be any permutation of the integers $\{1, 2, \dots, n\}$. If customers are served in the order I , the total time passed in the system by all the customers is

$$\begin{aligned} T &= t_{i1} + (t_{i1} + t_{i2}) + (t_{i1} + t_{i2} + t_{i3}) + \dots \\ &= n t_{i1} + (n-1) t_{i2} + (n-2) t_{i3} + \dots \\ &= \sum_{k=1 \text{ to } n} (n - k + 1) t_{ik} \end{aligned}$$

Optimality Proof (II)

- Suppose now that I is such that we can find 2 integers a and b with $a < b$ and $t_{ia} > t_{ib}$: in other words, the a th customer is served before the b th customer even though a needs more service time than b .
- If we exchange the positions of these two customers, we obtain a new order of service I' . This order is preferable because

$$T(I) = (n - a + 1)t_{ia} + (n - b + 1)t_{ib} + \sum_{\substack{k=1 \\ k \neq a, b}}^n (n - k + 1)t_{ik}$$

$$T(I') = (n - a + 1)t_{ib} + (n - b + 1)t_{ia} + \sum_{\substack{k=1 \\ k \neq a, b}}^n (n - k + 1)t_{ik}$$

$$\begin{aligned} T(I) - T(I') &= (n - a + 1)(t_{ia} - t_{ib}) + (n - b + 1)(t_{ib} - t_{ia}) \\ &= (b - a)(t_{ia} - t_{ib}) > 0 \end{aligned}$$

Example

- We have 3 customers with

$$t_1 = 5, \quad t_2 = 10, \quad t_3 = 3$$

Order: 1 3 2

$$T(I) = 5 + (5 + 3) + (5 + 3 + 10) = 3 \times 5 + 2 \times 3 + 1 \times 10 = 31$$

Order: 3 1 2

$$T(I') = 3 + (3 + 5) + (3 + 5 + 10) = 3 \times 3 + 2 \times 5 + 1 \times 10 = 29$$

$$T(I) - T(I') = 3 \times (5 - 3) + 2 \times (3 - 5) = (3 - 2)(5 - 3) = 2 > 0$$

Optimality Proof (III)

- We can therefore improve any schedule in which a customer is served before someone else who requires less service. The only schedules that remain are those obtained by putting the customers in non-decreasing order of service time. All such schedules are equivalent and thus they're all optimal.

Service Order

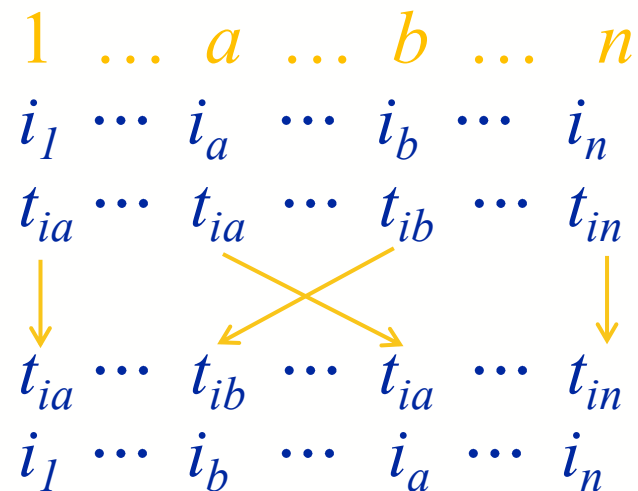
Served Customer

Service Duration

After exchange i_a & i_b

Service Duration

Served Customer

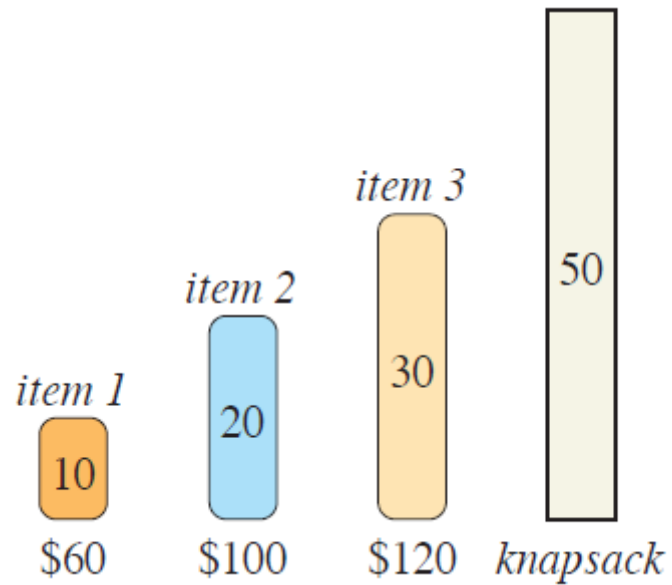


0/1 Knapsack

Search Algorithms

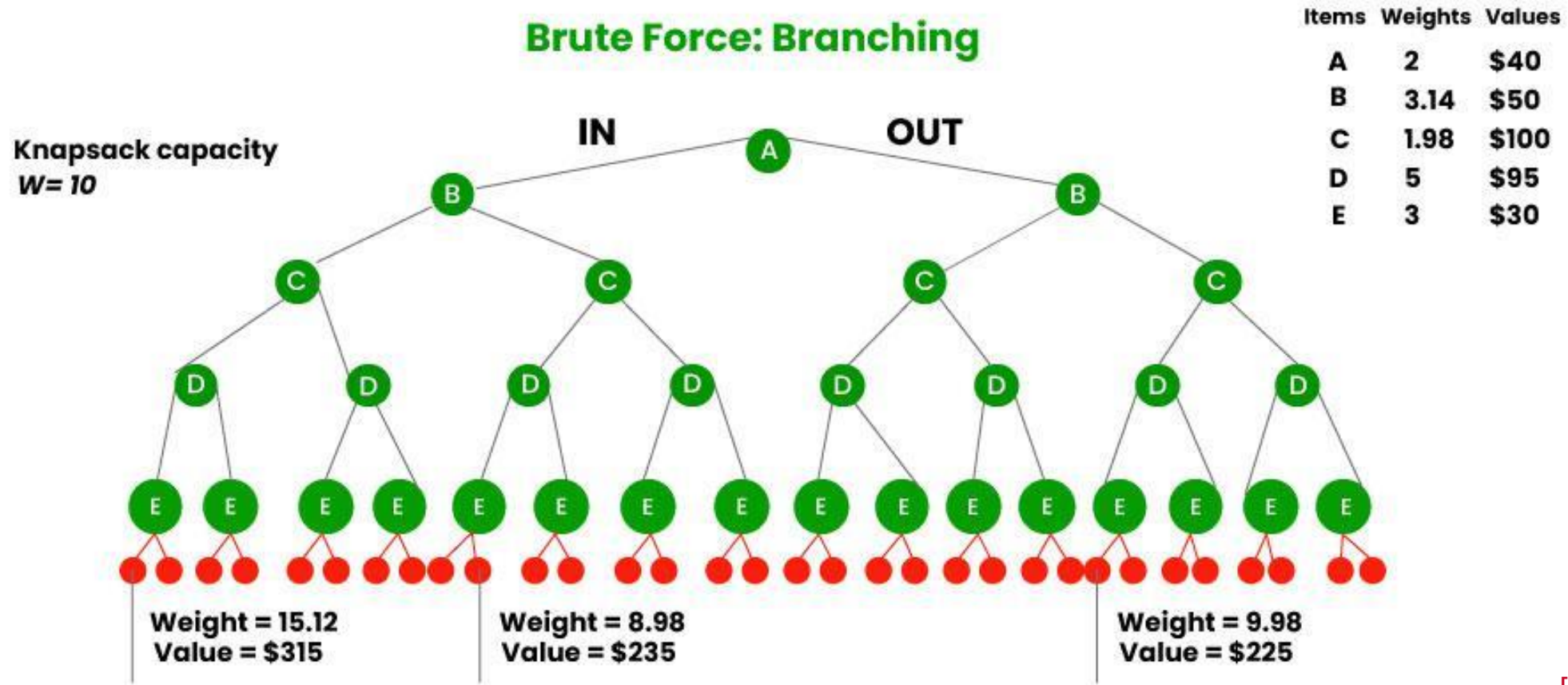
Knapsack Problem

- Objective: Maximize the total value of items placed in the knapsack without exceeding the weight capacity.
- **0-1 knapsack**: Binary decision for each item (take it or leave it)



item 1	item 2	item 3	total weight	total value
0	0	0	0	0
0	0	1	30	120
0	1	0	20	100
0	1	1	50	220
1	0	0	10	60
1	0	1	40	180
1	1	0	30	160
1	1	1	60	280

Brute Force (DFS)

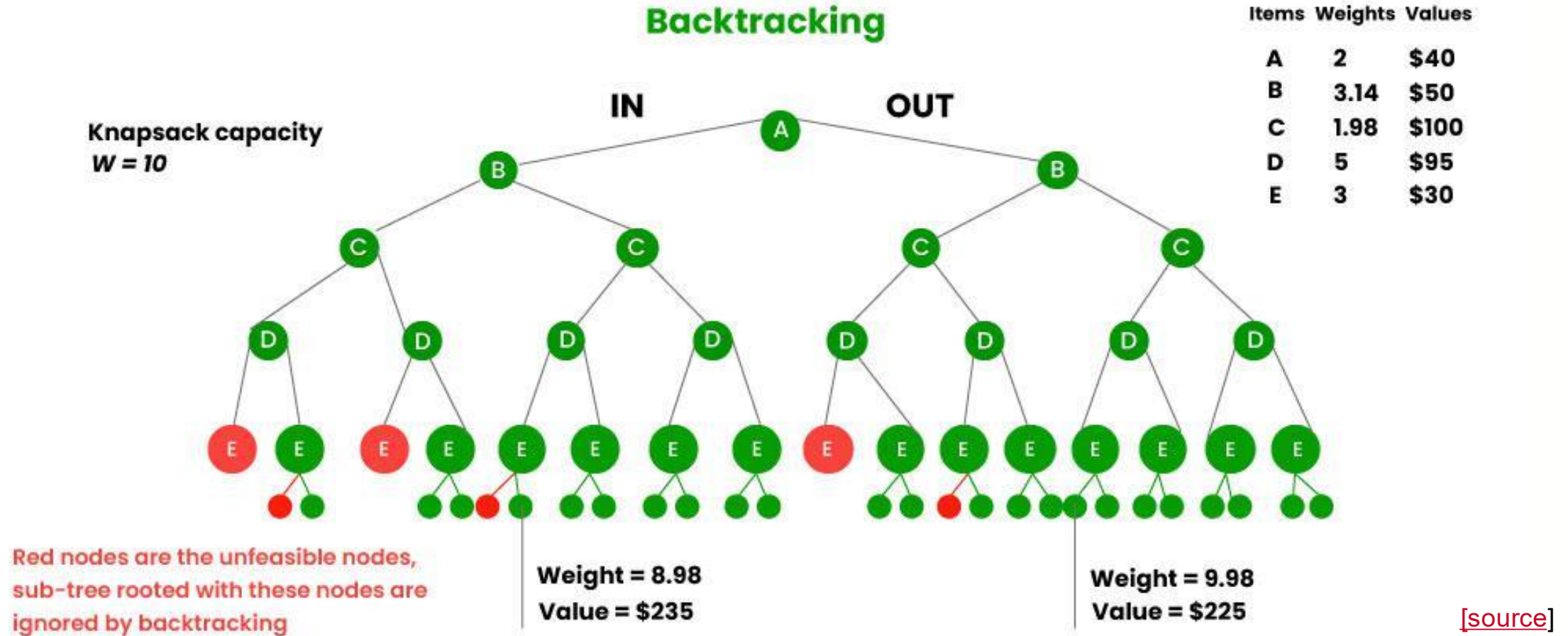


[source]

Knapsack using Branch and Bound



Backtracking



Knapsack using Branch and Bound



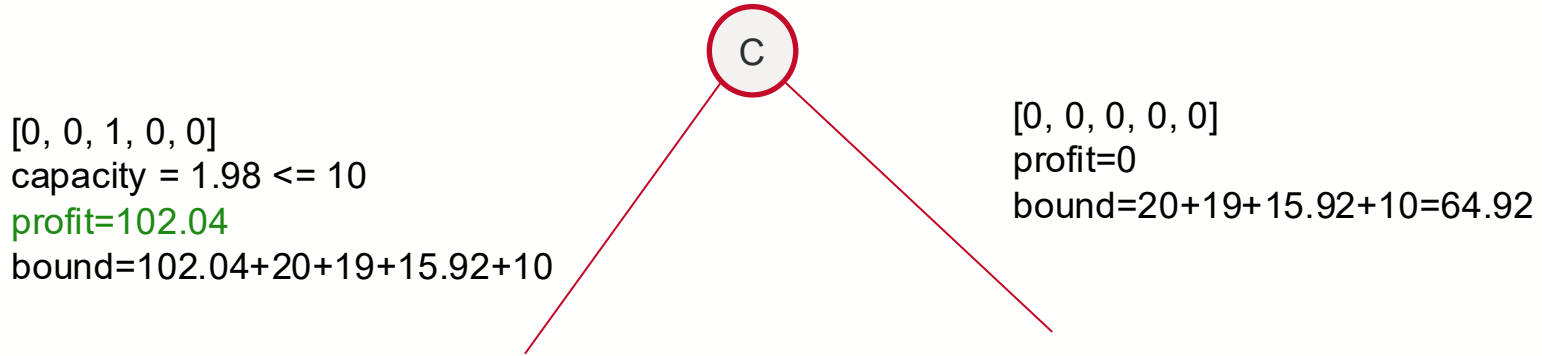
Branch and Bound

- Systematically explore the state-space tree using a “best-first” approach, pruning any branch that cannot mathematically beat the current best solution
- Key steps:
 - Sort items by value density (value/weight ratio) to compute a Greedy upper bound
 - Initialization
 - $\text{maxProfit} = 0$
 - Enqueue root node ($\text{profit} = 0$, $\text{weight} = 0$)
 - While queue not empty, dequeue the most promising node u and explore its children
 - Branch1 (include item)
 - feasibility test: $\text{currentWeight} + \text{itemWeight} \leq \text{Capacity}$
 - if $\text{profit} > \text{maxProfit}$, update it
 - if $\text{bound} > \text{maxProfit}$, enqueue the node # Greedy upper bound – fractional knapsack
 - Branch2 (exclude item)
 - if $\text{bound} > \text{maxProfit}$, enqueue the node # Greedy upper bound – fractional knapsack

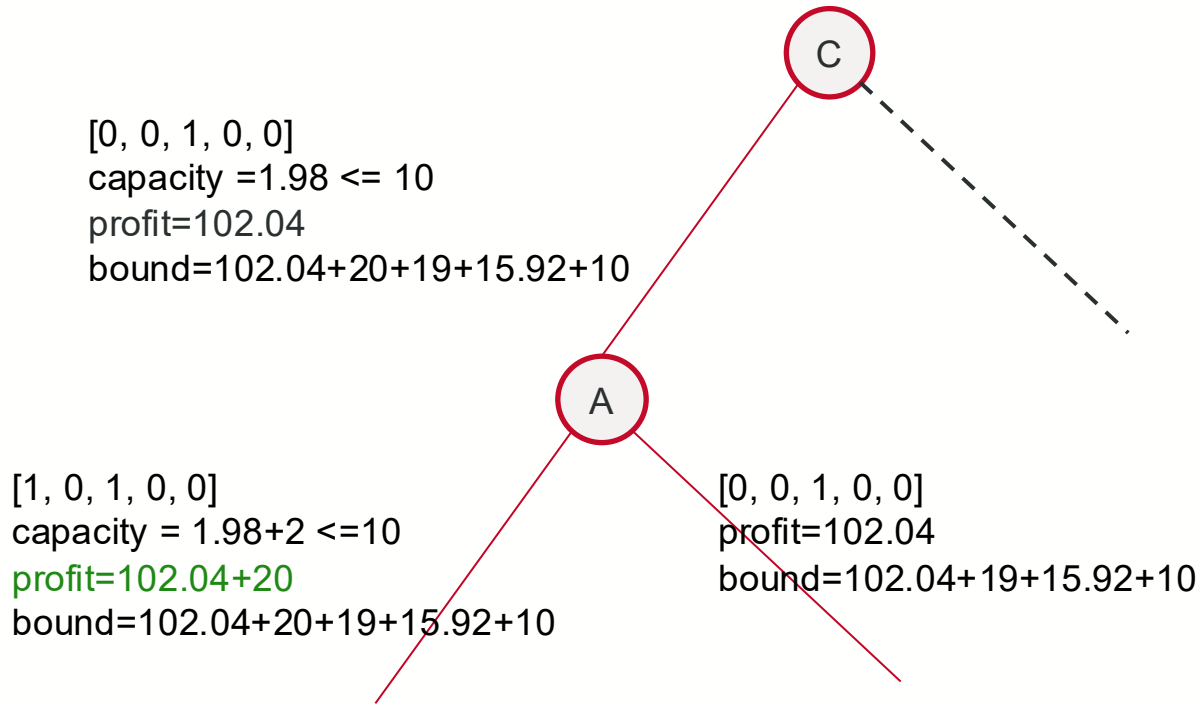
C

Input: $N = 5$, $W = 10$, $v[] = \{40, 50, 100, 95, 30\}$, $w[] = \{2, 3.14, 1.98, 5, 3\}$

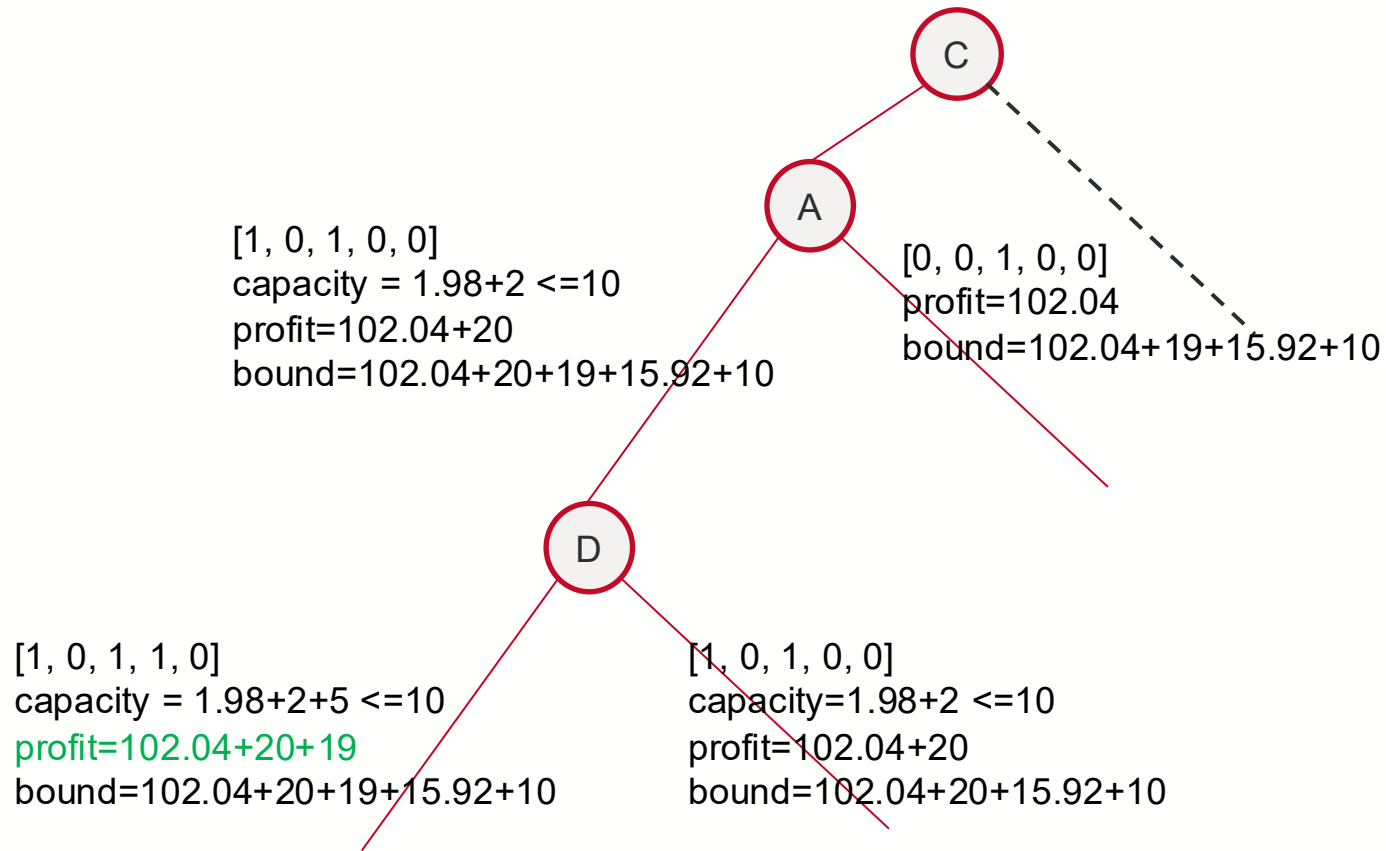
	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10



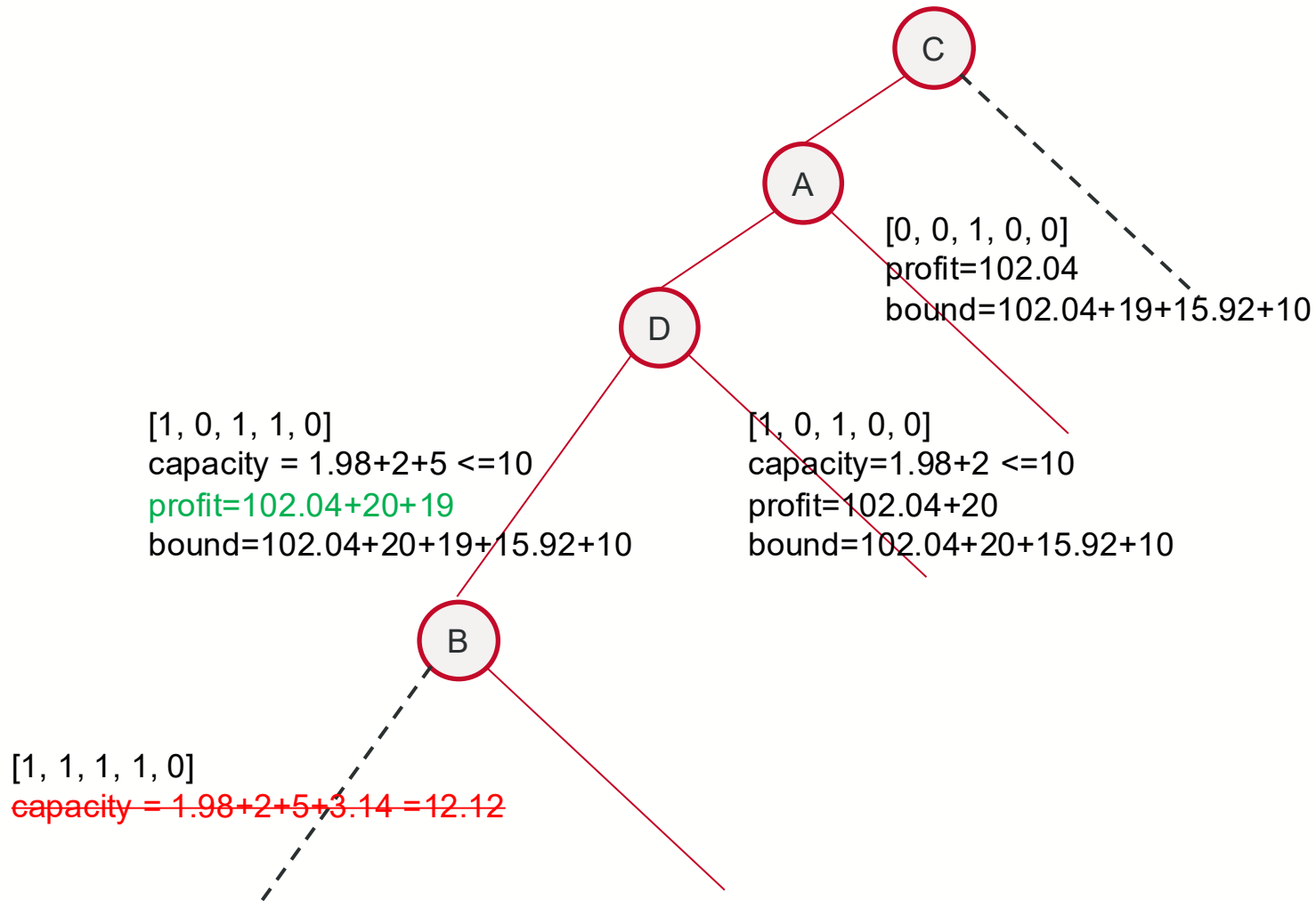
	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10



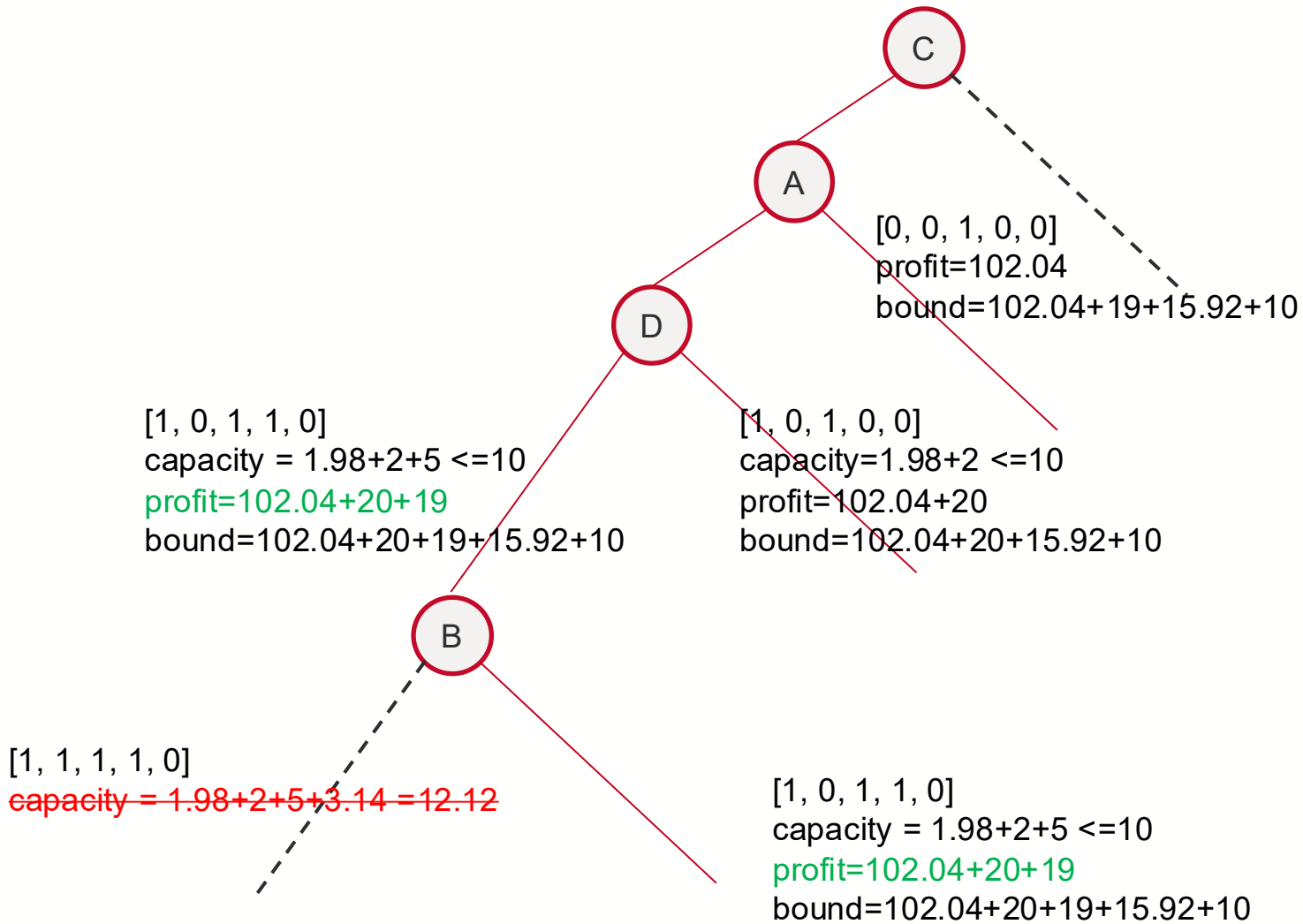
	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10



	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10



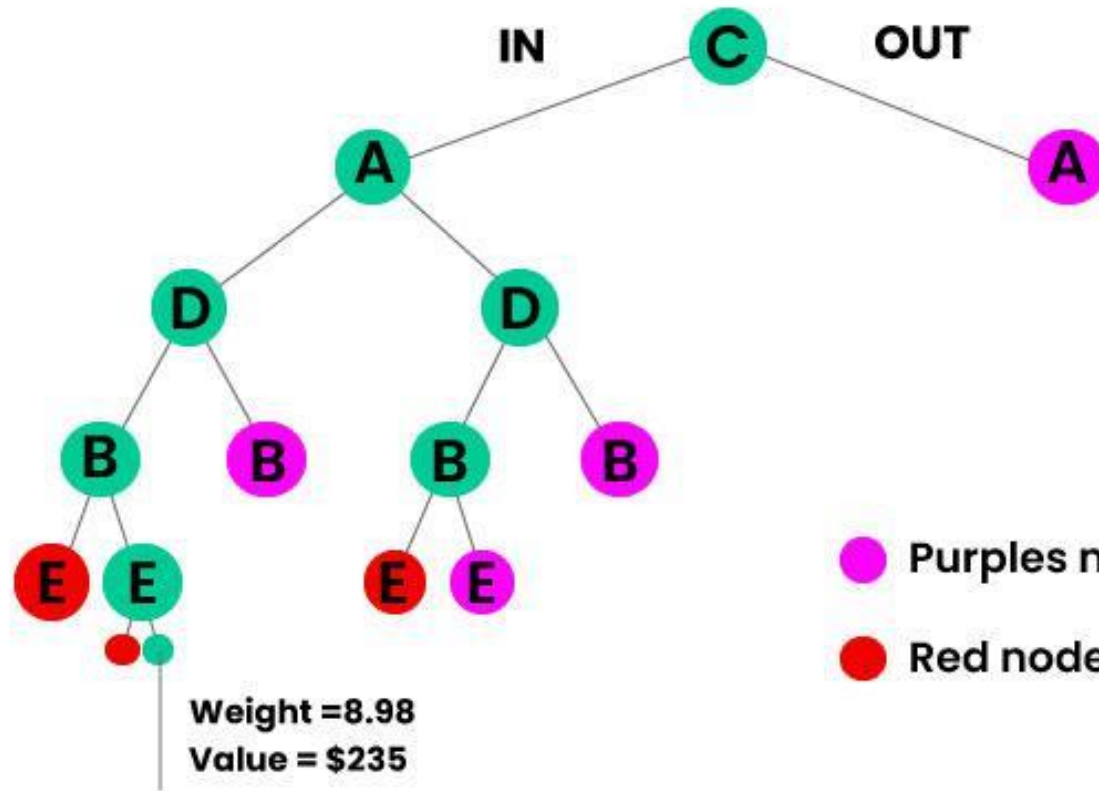
	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10



	Weight	Value Density
C	1.98	102.04
A	2	20
D	5	19
B	3.14	15.92
E	3	10

Try the rest yourself!

Branch and Bound



Items {Weights, Values }

A	{2, 40}
B	{3.14, 50}
C	{1.98, 100}
D	{5, 95}
E	{3, 30}

● Purples nodes are ignored because of bounds

● Red nodes are ignored because of Infesasibility

[\[source\]](#)

Knapsack using Branch and Bound

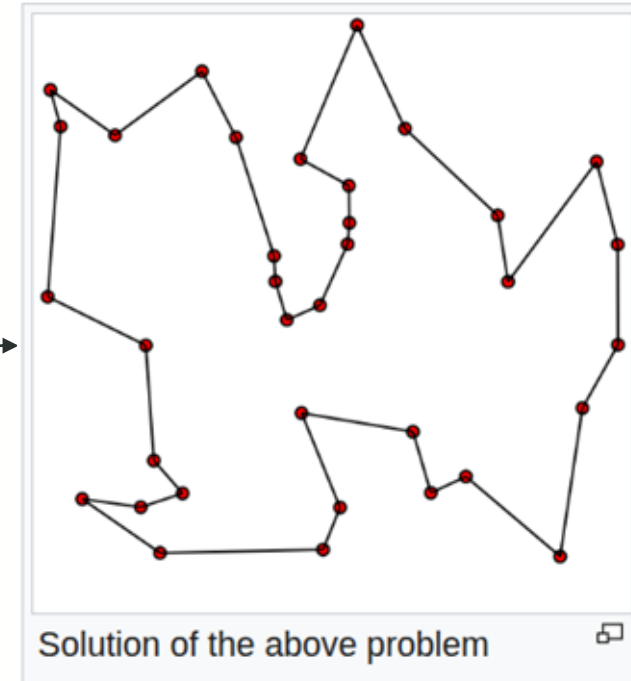
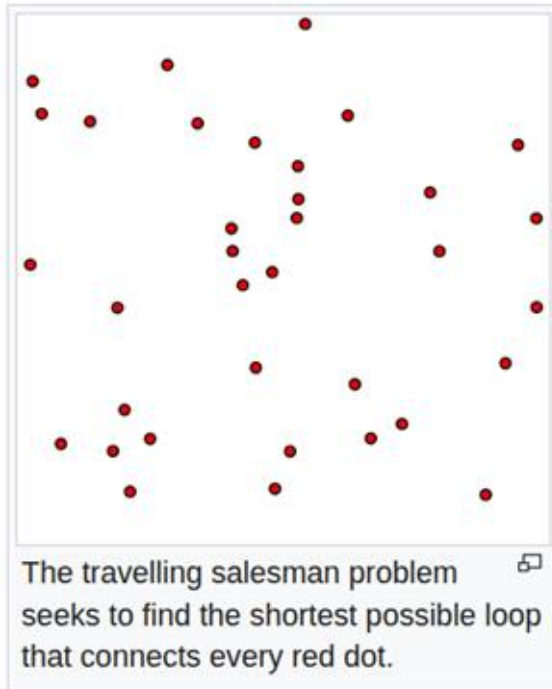


TSP

Greedy and Search Based Algorithms

TSP

- "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"



"Travelling salesman problem." Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Travelling_salesman_problem

Graph

- Visualize the graph corresponding to the adjacency matrix

	A	B	C	D
A	0	20	42	35
B	20	0	30	34
C	42	30	0	12
D	35	34	12	0

- Try to find the shortest route that visits each city exactly once and returns to the starting city

Properties

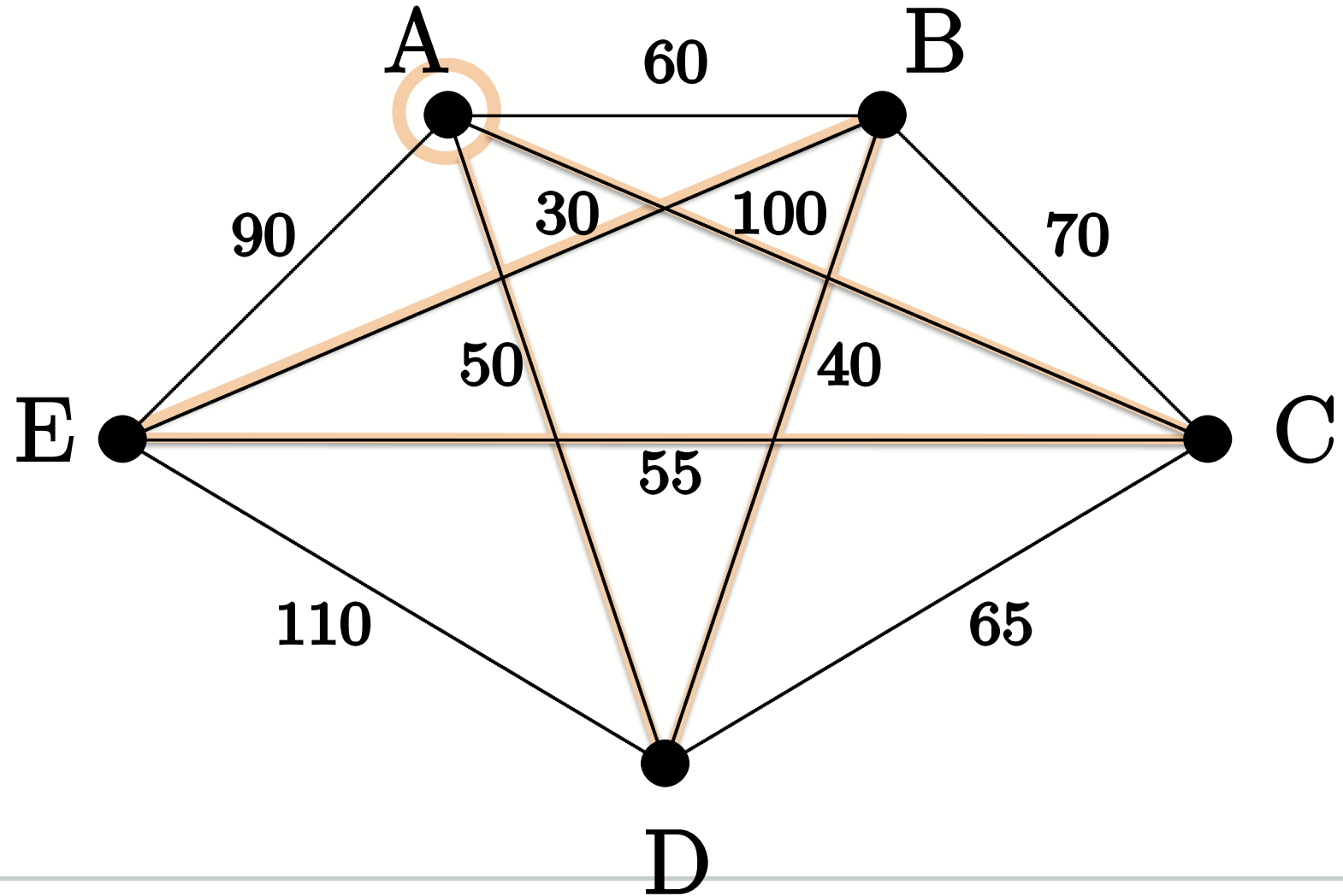
- Does TSP have optimal substructure?
 - If an optimal tour visits cities in the order: $A \rightarrow B \rightarrow C \rightarrow D$
 - then the subpath $B \rightarrow C \rightarrow D$ must also be optimal for visiting $\{B, C, D\}$ in that order
- Does TSP have the greedy-choice property?

Greedy

- Construction Heuristics (building the tour)
 - Nearest Neighbor (NN): Start anywhere, always pick closest unvisited city
 - Repeated NN: try NN from every starting city, pick best
- Improvement Heuristics (refining the tour)
 - Greedy 2-opt: A local search strategy that accepts the first improvement it finds
- What Greedy TSP is good for
 - A great baseline heuristic
 - Useful for initializing local search
 - Extremely fast for large instances
 - Often “good enough” in practice for rough approximations

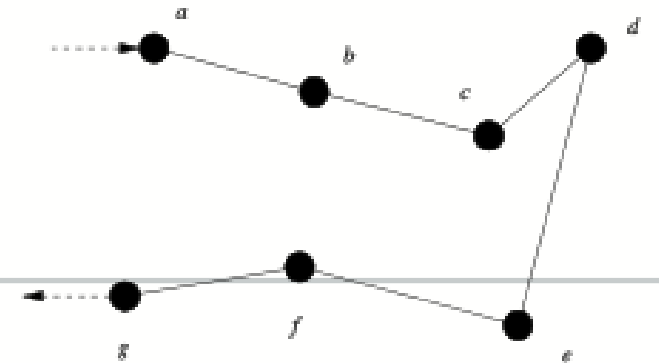
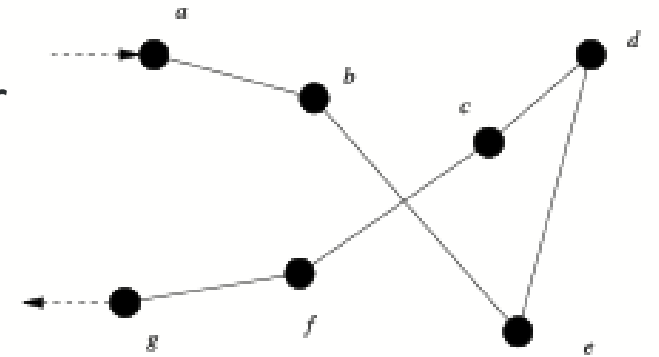
Nearest Neighbors: NN

- Route: A
 - A->B = 60
 - A->C = 100
 - A->D = 50
 - A->E = 90
- Route: A, D
 - D->B = 40
 - D->C = 65
 - D->E = 110
- Route: A, D, B
 - B->C = 70
 - B->E = 30
- Route: A, D, B, E
 - E->C = 55
- Route: A, D, B, E, C
- Route: A, D, B, E, C, A



2-OPT (Improvement heuristic, Local search method)

- Two TSP tours are called 2-adjacent if one can be obtained from the other by deleting two edges and adding two edges
- 2-opt repeatedly looks for a 2-adjacent tour with lower cost
 - Remove two edges from the tour
 - Reverse the segment between them and reconnect the tour
 - If the new tour is shorter, accept it
 - Repeat until no improving 2-opt move exists



Brute Force

- Generate all $(n-1)!$ possible tours, calculates the total distance for each, and picks the minimum
- Guaranteed to find the optimal solution
- However, practically impossible for even 20-30 items

TSP using Search Algorithms

- Naïve DFS (Brute force): visit every possible node to the maximum depth
- Backtracking: An optimized DFS that uses a feasibility test to prune branches as soon as they violate a constraint
 - If the current path you are building is already longer than the best complete tour you've found so far, you stop and backtrack
- Branch and bound: Add a greedy look-ahead (best-case future of a path) to make the pruning happen much sooner

Discussion – TSP Branch and Bound

- In the context of TSP, what would be a good look-ahead method to estimate the remaining distance?
- To prune effectively, do we need to calculate a lower bound or an upper bound on the remaining cost? Why?
- What is the hidden cost of a “perfect” look-ahead (bound)?
- Trade-off: Tighter bounds (pruning) vs. computational overhead
- If a tighter bound prunes 90% of the tree but takes 10x longer to calculate at every node is this better than not pruning at all?
 - hint: Consider the total execution time $T = \text{nodes visited} \times \text{time per node}$